# Collect & Adjust Server-Level Power Consumption

Daniel Lee
Grinnell College
Grinnell, Iowa, USA
leedanie2@grinnell.edu

***Abstract*—Over the past several years, advances in high-performance computing (HPC) have resulted in the proliferation of bulk-synchronous jobs. By nature, these workloads cause large fluctuations in cluster-level power consumption, which can damage on-site electrical infrastructure and the grid connection [1]. Battery energy storage systems can absorb these swings, but they add cost and increase the environmental footprint [1]–[3]. This paper evaluates our two software-only solutions across three workloads, with four repeats per measurement, on a dual-socket Xeon server. CPU power was sampled from Intel Running Average Power Limit (RAPL) energy registers at 10 Hz. A live dashboard stack displays server CPU power and supplementary telemetry data, using 0.017% of host server capacity for data collection. Both mechanisms run alongside HPC jobs and raise power draw for greater stability: 1) Ramp-and-fill gradually ramps power, sustains a power ceiling with secondary computation threads until the original job is finished, then ramps down to idle. Compared to baseline power data, this method reduced the coefficient of variation (CV) by $89.5 \pm 0.6\%$ while the primary job ran. The ramps add roughly 2.5 to 3 minutes per job, a fixed overhead amortized over long-running jobs with short power dips. 2) The governor reacts to rapid changes in CPU usage by spawning threads that limit the slew rate, reducing CV by $25.7 \pm 4.0\%$. In response to benchmarks, its two-threshold CPU-usage heuristic for dangerous power regime detection caught 99% of 557 labeled spikes, with 5 false alarms per hour. However, flagging did not precede the abrupt change. The dashboard stack and ramp-and-fill were delivered to internal NSA teams, and the governor and telemetry pipeline were shared with the NSA's research partner, AMD, to motivate lower-latency energy-reporting hardware. Improved software solutions require telemetry that can detect power swings before they begin.***

***Index Terms*—Data centers, power smoothing, power system stability, RAPL, telemetry**

## I. Introduction

In recent years, advances in artificial intelligence have transformed how data centers use power. While traditional data centers run numerous independent workloads in parallel, AI infrastructure has distinct qualities designed for bulk-synchronous jobs. As the name suggests, these loads require nodes to synchronize. To understand the implications of this model, one must recognize that bulk-synchronous loads consist of two alternating phases: computation and communication. During the computation phase, each node processes a specific data set. Once all calculations have concluded, the results must be aggregated through a period of inter-node communication [1].

One may ask how this affects power consumption. The answer is that the computation phase demands significantly more power than the communication phase. When many synchronized nodes alternate between these phases, the fluctuations compound in facility power consumption [1]. This variable power draw damages hardware, shortening the lifespan of microelectronic components and facility infrastructure [1], [4]. When the grid experiences significant fluctuations in power demand, it becomes vulnerable to instability, including harmonic distortion, frequency drift, and voltage flicker [1], [5], [6]. Furthermore, certain oscillation frequencies can induce resonance in generation equipment, accelerating fatigue [1].

To address these risks, engineers have developed low-loss energy storage devices that can absorb these fluctuations [1]. Choukse et al. [1] evaluated software, GPU hardware, and facility-level solutions for this problem scoped to large GPU training clusters. However, these hardware solutions require significant investment [1], making the software approach an appealing alternative. In this paper, we investigate the implementation and performance of two such approaches.

## II. Power Measurement

Intel's Running Average Power Limit (RAPL) is a hardware power-management architecture that maintains a running estimate of consumed energy across several processor domains: the CPU package, cores (PP0), uncore/graphics (PP1), and DRAM. It uses Model Specific Registers (MSRs) to record energy in a counter up to once per millisecond [7]. We calculate power as a running average by dividing the RAPL energy (in microjoules) between two reads by the wall-clock time that has elapsed. Rather than directly reading from the MSRs, we used the Linux powercap sysfs interface for portability [8]. Since a modern server is multi-socket, server energy is the sum of all package domains. RAPL's `energy_uj` is an accumulating counter that wraps at `max_energy_range_uj`. On Mycroft, an HPE server with Xeon CPUs, the bound is $\approx$262 kJ per package. Our script subtracts two consecutive readings and, for negative differences, adds `max_energy_range_uj` to guarantee positive energy deltas.

Although RAPL can read the energy counter at 1 kHz [7], we experimentally observed that sampling faster than 10 Hz on Mycroft produced phantom power spikes. We know these kilowatt-order spikes are not real because Mycroft's combined burst limit ($2 \times$ PL2) is 492 W. The false spikes occur because, at high rates, the energy counter does not advance on every read, allowing a series of unchanged reads followed by one read that captures all the accumulated energy. Because that delta is wrongly attributed to a single sample interval

TABLE I
POWERCAP CONSTRAINTS

| sysfs file | Name | Meaning | Mycroft |
|---|---|---|---|
| `constraint_0_power_limit_uw` | PL1 (long_term) | Sustained ceiling | 205 W |
| `constraint_0_time_window_us` | PL1 window | Averaging window for PL1 | — |
| `constraint_1_power_limit_uw` | PL2 (short_term) | Burst ceiling | 246 W |
| `constraint_1_time_window_us` | tau | Burst duration before clamp to PL1 | ∼1.6 s measured |

TABLE II
RAPL VS. REDFISH

| | RAPL | Redfish |
|---|---|---|
| Scope | CPU package, cores, DRAM | Whole chassis |
| Granularity | Per-socket, per-domain | Chassis total |
| Useful rate | ∼10 Hz | ∼1 Hz |
| Path | In-band, host CPU | Out-of-band, BMC |

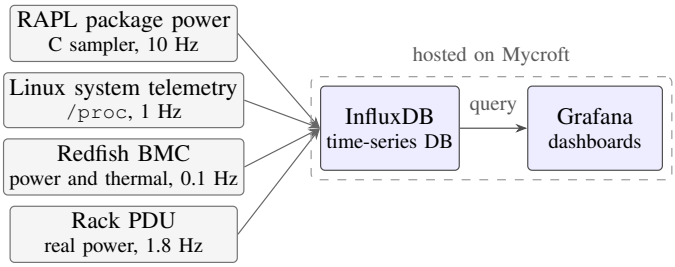



Fig. 1. Monitoring stack. Telemetry daemons write to InfluxDB on Mycroft, which Grafana queries for live dashboards.

TABLE III
SAMPLER-INDUCED IDLE NOISE, 120 S AT 10 HZ

| Condition | Mean (W) | Stdev (W) | Min (W) | Max (W) |
|---|---|---|---|---|
| Nothing running | 199.60 | 1.68 | 194.89 | 204.99 |
| `rapl_hf_sampler` (C) | 199.69 | 1.68 | 195.20 | 205.82 |
| `influx2.py` (Python) | 199.92 | 1.84 | 194.55 | 205.92 |

rather than the total duration of those reads, RAPL incorrectly reports a power spike. We fixed this in the live sampler by only reporting a power value when the counter advanced, dividing the energy delta by the time since the last advance. For Mycroft, we found that sampling above 10 Hz does not yield additional physical information.

### A. Power Capping

RAPL can also enforce power limits through the constraints in the table below [8]. The hardware requires PL2 $\geq$ PL1. On Mycroft, the CPU defaults are PL1 = 205 W and PL2 = 246 W. The Mycroft CPU maintains PL2 for a measured tau of $\sim$1.6 s, then steps down to the $\sim$205 W sustained floor. We exploited and worked around this clamping behavior to design power-ramping mechanisms.

Although Distributed Management Task Force (DMTF) Redfish [9] uses the industry standard API, it can only poll power up to once per second. We chose RAPL over Redfish as our primary power telemetry source to prioritize a higher sampling rate and domain granularity.

Because Redfish reports power for the whole chassis, we integrated Redfish to collect system-wide power and thermal management data. Redfish is located on the Baseboard Management Controller (BMC), a separate processor on the motherboard [9]. This means its polls have longer round-trip latency and impose less CPU overhead than RAPL, which uses the host CPU to read and send samples. The powercap interface is compatible with AMD's RAPL registers [10], so the same approach applies to AMD hardware running Linux.

## III. MONITORING STACK

To monitor Mycroft's live CPU power over time, we wrote a daemon to poll RAPL and compute mean power at a fixed rate. Our code streams data from RAPL into an InfluxDB time-series database, which Grafana queries for visualization. On Grafana, we built dashboards for CPU power, clock frequency, utilization, and temperature. We also had dashboards sourced from daemons for Linux telemetry, Redfish BMC, and the Power Distribution Unit (PDU). InfluxDB and Grafana are maintained on Mycroft as background daemons. Hosting on Mycroft as opposed to another server allowed all project contributors to access our monitoring tools.

Mycroft CPU power is sampled at 10 Hz while system, Redfish, and PDU daemons measure at 1 Hz, 0.1 Hz, and 1.8 Hz, respectively. The Redfish script is constrained far below 1 Hz because it makes multiple calls to collect other data along with power. PDU sampling is capped at 1.8 Hz by HTTP response time. Along with RAPL, we polled the Redfish BMC and the PDU for power, frequency, Power Supply Unit (PSU) voltage, temperature, and cooling power. Linux system telemetry provided disk activity, memory, load averages, context switches per second, and run-queue depth. Later, we ported some daemons from Python to C for lower Mycroft overhead.

When monitoring power, we aimed for minimal operational overhead to collect cleaner data from running workloads and reduce the impact of confounding variables. We found that the Python RAPL script used 2.34% of one core and 50.4 MiB. The C script, which collected RAPL per-core statistics and 64 temperature sensors, used only 0.38% of a core and 2.3 MiB. This lowered the standard deviation of idle package power from 1.84 W with Python to 1.68 W using C, as shown in Table III. The standard deviation for C was the same as when nothing was running, implying that C imposed minimal overhead.

We also analyzed our full monitoring stack using 60 s windows for an idle Mycroft. Altogether, our collection tools cost 2.19% of one core, or 0.017% of the server's capacity, and 443 MiB, excluding the InfluxDB storage layer. InfluxDB raises the idle CPU power by 6.0 W even without writers, which costs more than the entire collection layer. From running the 10 Hz RAPL writer on top of InfluxDB, we found

TABLE IV
TELEMETRY STACK OVERHEAD (MYCROFT, IDLE, 60 S WINDOWS)

| Process | Poll rate | CPU (% of 1 core) | RSS (MiB) |
|---|---|---|---|
| `rapl_hf_sampler` (C) | 10 Hz | 0.38 | 2.3 |
| `influx2.py` (Python) | 10 Hz | 2.34 | 50.4 |
| Redfish poller | 0.1 Hz | 0.43 | 56.1 |
| PDU poller | ∼1.8 Hz | 0.47 | 26.2 |
| Grafana 13.0.2 | idle | 0.91 | 357.7 |
| Collection layer total* | | 2.19 | ∼443 |
| InfluxDB (no writers) | | 6.0 (settled) | 396 (settled) |

*C sampler + Redfish + PDU + Grafana; 0.017% of 128 threads.

that RAPL costs 0.15 W and 0.41% of one core. Similarly, deploying Grafana on those processes increased power by 4.44 W. From the data, we can infer that the cost of in-band telemetry is due more to storage than to polling frequency.

We cross-checked our dashboards against system-wide telemetry using `top` and `lscpu`. From our visualization dashboards, we observed several relationships between data, particularly the close dynamic between CPU utilization and package power.

## IV. Power-Smoothing Mechanisms

A common approach in the literature to the power swing problem is to deploy secondary workloads when the primary job is finished [1]. The secondary workload can be anything that burns power to prevent large drops or upward spikes. We evaluate two mechanisms built on this idea: ramp-and-fill and the governor.

### A. Ramp-and-Fill

Ramp-and-fill (`ramp.c`) linearly builds up and winds down a specified number of cores at a predetermined rate through CPU usage. In practice, we ramped up the power to max, allowed the workload to run, then ramped down after completion. During the workload, filling strives to keep computational resources active throughout the entire duration of a primary task. To accomplish this, the workers are pinned to the same logical CPU cores as the primary task. Whenever the primary task stops utilizing the full extent of its allocated resources, the Linux scheduler engages workers, which perform floating-point arithmetic operations. In order to minimize the performance impact of the workers, they are given the scheduling priority `SCHED_IDLE` [11], forcing them to immediately yield CPU time at the request of the primary task.

### B. Governor

To build a solution that detects both power spikes and drops, the governor was created. The governor uses `usage_edge`, which replaced all previous ML methods for power swing detection. The benefit of using this detector is that it does not rely on power, meaning that it is not restricted to confirming that the power has changed. It is not necessarily predictive. However, the high correlation between power and usage inspired the `usage_edge` and governor stack. We use kernel counters via `/proc/stat` to monitor the usage every 100 ms

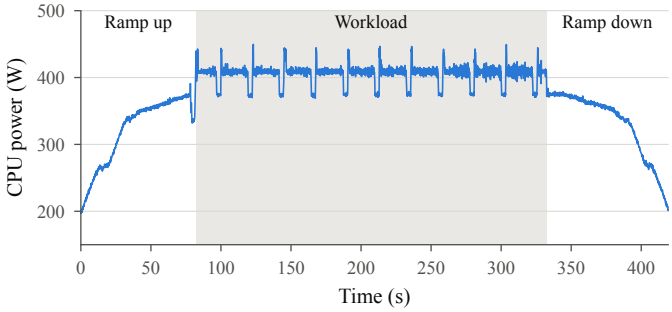



Fig. 2. Ramp-and-fill power profile: ramp up, workload, and ramp down. Measured RAPL CPU package power at 10 Hz for HPL (run 1); the shaded span is the trimmed workload window.

TABLE V
GOVERNOR PARAMETERS

| Parameter | Value | Controls |
|---|---|---|
| Detector poll interval | 100 ms | `/proc/stat` sampling |
| Usage threshold | 20% | Edge definition |
| Usage slope | 25 percentage points/s | Rise or fall |
| Flag durations | 1 s high, 5 s cooldown | Flag duration and spacing |
| Control cadence | 50 ms | Governor loop |
| Slew rate | 80 W/s | Trigger |
| Ballast pool | 64 cores at 3.26 W/core | Capacity |

(10 Hz). Each consecutive pair of samples is computed as a rate. If (1) the interval went from below to above 20% or (2) the rate is above 25 percentage points per second, we flag the event as an edge. Depending on whether it is positive or negative, the edge is declared either a rise or a fall. The edge is flagged for 1 second, and the next flag can be fired after 5 seconds to prevent repeated alarms.

RAPL power caps are employed against spikes. For drops, we use a secondary workload. When a rise is detected, we impose a RAPL ceiling 55 W higher than the peak and allow power to climb at a rate that guarantees the ramp-up phase is 3 seconds long. Since a power ceiling cannot prevent a fall, we incorporate a ballast technique. Inspired by ramp-and-fill, the Linux scheduler assigns jobs from a pool of duty-cycled spinner processes at `SCHED_IDLE` to burn power. All cycles are paused when the primary workload returns. The governor also has an engage gate, which requires that any smoothing action is only triggered by a transition of 80 W/s or a 45 W jump between polls. Having a strict condition for action means high precision and low overhead but leaves the governor vulnerable to inaction.

## V. Experimental Setup

After killing processes on Mycroft to quiesce background noise, we ran three distinct workloads as baselines and compared them to each mechanism running the corresponding workload. We accomplished this through CSV collection, repeating each run four times. In total, we had one baseline and two mechanisms for each workload, meaning nine CSVs across each trial and 36 traces overall. Mycroft uses dual Xeon

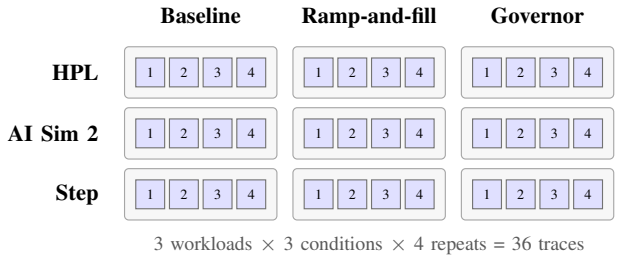



Fig. 3. Run matrix: 3 workloads × 3 conditions × 4 repeats.

Gold 6338 CPUs with 128 threads [12]. The idle power is around 200 W and the sustained limit (2 × PL1) is 410 W.

To test our solutions against a variety of jobs, we designed three workloads with unique power profiles. We employed a mix of oscillations between idle and max power, random peak widths, and random step-like sustained draw.

### A. HPL

Our first workload uses HPL to mimic AI training by switching between intensive compute phases and idle communication periods. On Mycroft, the baseline workload takes about 18 seconds at high power and 3 seconds at low power. It runs HPLinpack 2.3 [13] with matrix order $N = 80{,}000$, block size 192, and 128 MPI ranks on an $8 \times 16$ grid, with dips imposed by SIGSTOP/SIGCONT per rank and a 3 ms stagger. We chose it because of its ability to recreate a simple, realistic HPC job.

### B. AI Sim 2

This is a Python benchmark that simulates sharp CPU loads and power consumption of an AI/LLM inference workload. It spawns one worker per logical CPU core. The workers alternate between a rest phase, a short sleep loop drawing minimal power, and a prefill phase, a continuous loop of dense floating-point math with all cores at 100% utilization. The benchmark partitions a 120 s window into interleaved rest and prefill segments, featuring three burst events with a minimum duration of 5 seconds each. A random seed is used to allow exact reproduction of the schedule across runs. As soon as a prefill phase ends, workers stop the math execution and drop to idle, producing a steep power drop.

### C. Step

This programmable workload creates a specified number of worker threads (124 in our runs) and reads a waveform file. Each instruction in the file stipulates that worker threads should maintain a particular CPU utilization over a prescribed duration, achieved through a work-sleep routine. Since CPU utilization and power consumption are closely correlated, the program is an effective generator of power consumption profiles.

TABLE VI
STEP WAVEFORM

| Instruction | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| Time (s) | 15 | 10 | 15 | 10 | 10 | 15 | 15 | 10 | 10 |
| CPU utilization (%) | 50 | 0 | 100 | 50 | 0 | 50 | 100 | 50 | 0 |

TABLE VII
CV REDUCTION (%) BY WORKLOAD AND MECHANISM, MEAN ± SAMPLE SD OVER FOUR RUNS. RAMP-AND-FILL USES THE TRIMMED WINDOW; THE GOVERNOR USES THE FULL TRACE. OVERALL AVERAGES THE THREE WORKLOADS WITHIN EACH RUN.

| Mechanism | HPL | AI Sim 2 | Step | Overall |
|---|---|---|---|---|
| Ramp-and-fill | $84.7 \pm 1.9$ | $88.0 \pm 0.3$ | $95.8 \pm 0.1$ | $89.5 \pm 0.6$ |
| Governor | $25.9 \pm 7.0$ | $44.9 \pm 7.3$ | $6.4 \pm 0.6$ | $25.7 \pm 4.0$ |

### D. Metrics

We scored power smoothing with the coefficient of variation, $\mathrm{CV} = \sigma(P)/\mu(P)$ of package power, and report reductions relative to the paired baseline. For ramp-and-fill, we trimmed the Mycroft traces to the window where the workload actually runs. This is to have an even comparison for load volatility. The window spans the first to last crossing of 99% of the median of the upper half of power samples.

## VI. RESULTS

Over the trimmed window, Ramp-and-fill reduced CV by $89.5 \pm 0.6\%$, which breaks down by workload as: HPL 84.7%, AI Sim 2 88.0%, step 95.8%. The Governor was evaluated over the full trace and achieved a substantial CV reduction of $25.7 \pm 4.0\%$. Per workload, it scored HPL 25.9%, AI Sim 2 44.9%, step 6.4%. In response to HPL, the detector built into the Governor caught 552 of 557 spikes with 5.2 false alarms per hour. This usage-based detector had a median lag of 0.23 s on AI Sim 2. In comparison, the previous random forest detector consistently responded to swings 10 to 20 s after onset, with about 141 false alarms per hour. These results convinced us to replace the more intricate random forest model with the faster two-threshold usage detector, which also produced fewer false alarms. The traces were recorded with a collector lacking the counter-advance fix, but removing artifact samples changed the CV reductions by less than 0.8 percentage points.

## VII. DISCUSSION

One big difference between the approaches is that Ramp-and-fill is proactive while the Governor is reactive. Ramp-and-fill maintains near-constant power during the workload but requires careful ramping before and after. This means running a job with this mechanism adds ramp-up time and power cost for both ramps. These drawbacks are amortized when the desired job lasts much longer than the ramps. Because the cost of sustaining a high power during execution depends on the distance from the elevated baseline power, this marginal cost is minimized for jobs that spend most of the time near peak power with brief drops, such as AI training [1].

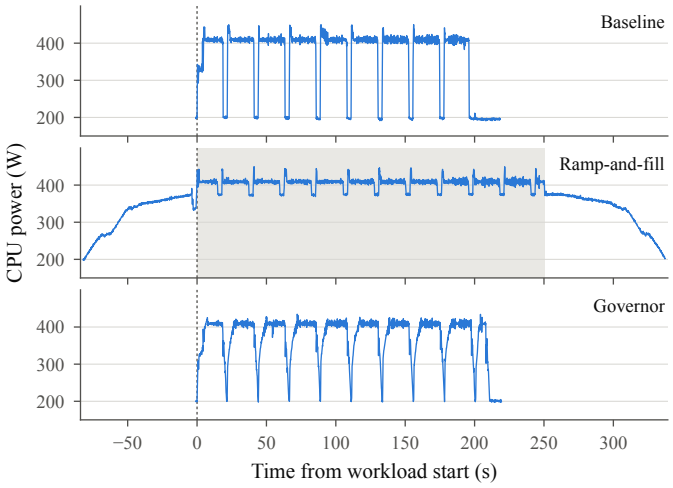



Fig. 4. Measured RAPL CPU package power at 10 Hz for HPL (run 1) under baseline, ramp-and-fill, and the governor, aligned to workload start (dashed line). The shaded span is ramp-and-fill's trimmed window.

Conversely, the Governor must detect before executing a secondary workload to smooth the draw, so the additional cost largely comes when it reacts. Compared to Ramp-and-fill, the Governor incurs substantially less cost in power and, in theory, its cost should not be uniquely suited to certain workloads. The Governor scored low on CV reduction for the step workload because CV is not a good proxy for this workload. A reactive smoother only addresses the transitions between power levels, and most of the variation occurs from distinct sustained power levels rather than erratic, spiky behavior.

The cost of employing the two solutions can be evaluated through the ratio of areas under the time-series power graphs of a workload with and without a software mechanism. Because this cost comparison is specific to the running workload, and we did not have access to realistic HPC jobs that take much longer than our workloads of 2 to 5 minutes, we omit the numbers that accompany our CV results. Similarly, we can identify performance cost by comparing runtimes, but we do not consider these corresponding numbers to be representative of HPC jobs. All data are available in the GitHub repo linked in the conclusion.

Finally, we note that our insights may be scoped to a single server. The power that we were measuring was only from the CPU package through RAPL. Ramp-and-fill's "trimmed window," where the primary workload was supposed to be executing, was determined through a heuristic.

## VIII. Conclusion

The governor's limitations can largely be attributed to its reactive approach. Since it does not employ corrective measures until after a large fluctuation is detected, it cannot mitigate its initial effects. Furthermore, actions taken to alleviate power swings invariably affect the measurements used for their detection, creating numerous issues that stem from self-reference. The software approach is best understood as an alternative, reserved for scenarios where hardware solutions cannot be employed. Future work includes porting the RAPL tooling to Ramius, an AMD-based HPC cluster, to compare cap behavior and overshoot dynamics. The repository (https://github.com/leedanie2/Collecting-and-Adjusting-Server-Power) also contains a facility grid model, a forecasting model, and a Ramp Down smoother, which were not evaluated in this paper.

## Acknowledgments

This work was supported by the 2026 Future Computing Summer Internship hosted by the Laboratory for Physical Sciences (LPS). The author was an ORISE participant affiliated with the NSA. A. Johnson developed ramp-and-fill. Erin Carrick and Stewart Baker mentored this work.

## References

[1] E. Choukse, B. Warrier, S. Heath *et al.*, "Power stabilization for AI training datacenters," arXiv:2508.14318, Aug. 2025. [Online]. Available: https://arxiv.org/abs/2508.14318

[2] W. Cole and A. Karmakar, "Cost projections for utility-scale battery storage: 2023 update," National Renewable Energy Laboratory, Golden, CO, USA, Tech. Rep. NREL/TP-6A40-85332, Jun. 2023.

[3] J. F. Peters, M. Baumann, B. Zimmermann, J. Braun, and M. Weil, "The environmental impact of Li-Ion batteries and the role of key parameters – A review," *Renewable and Sustainable Energy Reviews*, vol. 67, pp. 491–506, 2017.

[4] J. Srinivasan, S. V. Adve, P. Bose, and J. A. Rivers, "The case for lifetime reliability-aware microprocessors," in *Proc. 31st Annual International Symposium on Computer Architecture (ISCA)*, 2004, pp. 276–287.

[5] "IEEE recommended practice for monitoring electric power quality," IEEE Std 1159-2019 (Revision of IEEE Std 1159-2009), 2019.

[6] P. Kundur, *Power System Stability and Control*. New York, NY, USA: McGraw-Hill, 1994.

[7] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "RAPL in action: Experiences in using RAPL for power measurements," *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 3, no. 2, pp. 9:1–9:26, 2018.

[8] "Power capping framework," The Linux Kernel documentation, accessed: Nov. 1, 2026. [Online]. Available: https://docs.kernel.org/power/powercap/powercap.html

[9] DMTF, "Redfish specification," DMTF Standard DSP0266, version 1.24.0, Apr. 2026. [Online]. Available: https://www.dmtf.org/sites/default/files/standards/documents/DSP0266_1.24.0.pdf

[10] V. Ding, "powercap: Add AMD Fam17h RAPL support," Linux kernel commit 43756a298928, Nov. 2020. [Online]. Available: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=43756a298928c9a4e8201baaeb205c0c103728af

[11] "sched(7): Overview of CPU scheduling," Linux man-pages project, accessed: Nov. 1, 2026. [Online]. Available: https://man7.org/linux/man-pages/man7/sched.7.html

[12] Intel Corporation, "Intel Xeon Gold 6338 processor (48M cache, 2.00 GHz): Specifications," accessed: Nov. 1, 2026. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/212285/intel-xeon-gold-6338-processor-48m-cache-2-00-ghz/specifications.html

[13] A. Petitet, R. C. Whaley, J. Dongarra, and A. Cleary, "HPL – a portable implementation of the high-performance Linpack benchmark for distributed-memory computers," Version 2.3, Dec. 2018. [Online]. Available: https://netlib.org/benchmark/hpl/